

ECE391

Computer System Engineering

Lecture 2

Prof. Kirill Levchenko and Dong Kai Wang

Slides adapted Prof. Kalbarczyk and Lumetta

University of Illinois at Urbana- Champaign

Spring 2024

Aministrivia

- **MP0**
 - Due in TAs office hours by next Wednesday 1/24.
 - Demo in ECEB 3026 lab.
- **Mediaspace**
 - <https://mediaspace.illinois.edu/channel/channelid/330756922>
 - Recordings should be automatically uploaded in evening.
- Lecture and discussion slides will also be posted on course website (syllabus page).
- Future: please read course notes ahead of lecture.
 - Also check the various [x86 references](#).

Lecture Topics

- x86 instructions
- Operate instructions
- Data movement instructions
- Conditional codes
- Control flow instructions (if time allows)
- Assembler conventions
- Two five-minute assembly exercises (talk to your neighbors!)

Goal: After this lecture, you should be able to write tasks in x86 assembly.

Introduction and Basics

- What is x86? (Also called Intel 32-bit (IA-32) architecture)
 - variable-length instruction encoding (1-16 bytes)
 - unlike simple RISC ISAs with fixed encoding length
 - small register set: 8 mostly general-purpose
 - 32-bit, byte-addressable address space
 - complex addressing modes
 - furthermore, x86 instructions can directly have data in memory as operands
 - many data types supported by hardware
 - trivia question: why is it called x86?
 - due to backwards compatibility a *word* still has length of 16 bits

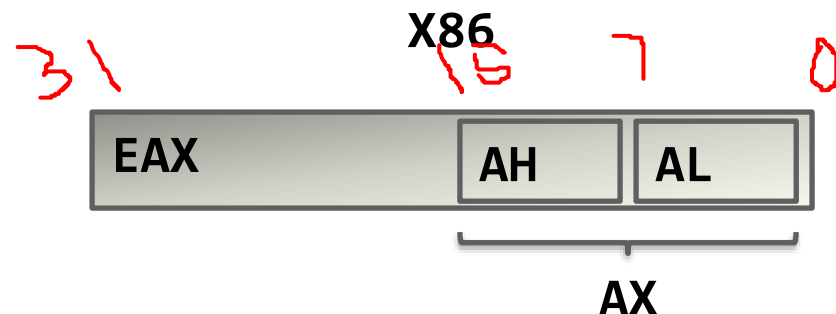
Registers

- Let's first see some general-purpose registers in x86:

R0
R1
R2
R3
R4
R5
R6
R7

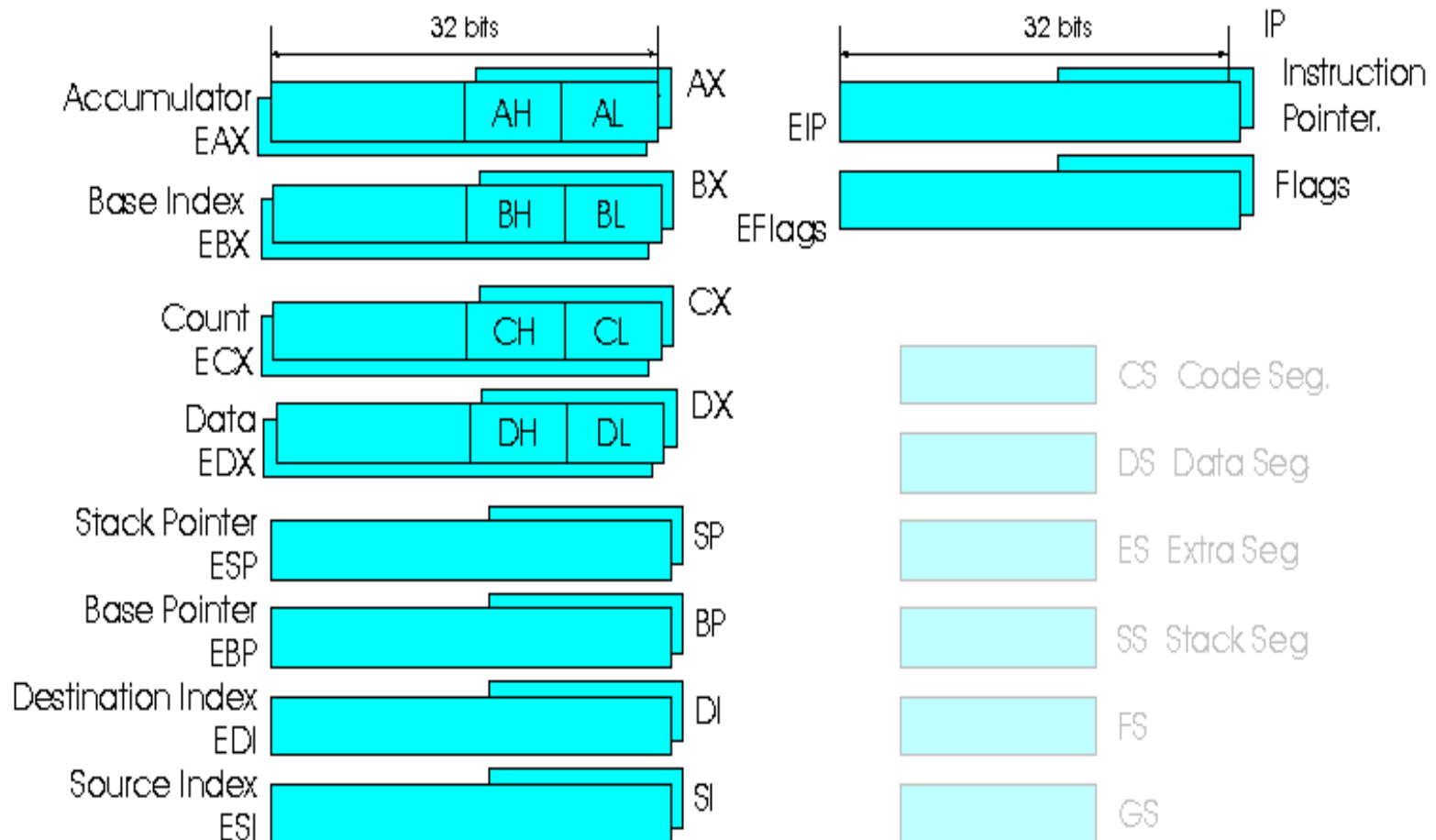
LC3 (16b)

A (Accumulator)
B (Base Index)
C (Counter)
D (Data)



Registers

General purpose registers



Registers



-----> *extended, i.e., 32-bit*

EAX accumulator	EIP instruction pointer
EBX base (of array)	EFLAGS flags/condition codes
ECX count (for loops)	
EDX data (2 nd operand)	
ESI source index (string copy)	
EDI destination index	
EBP base pointer (base of stack frame)	
ESP stack pointer	

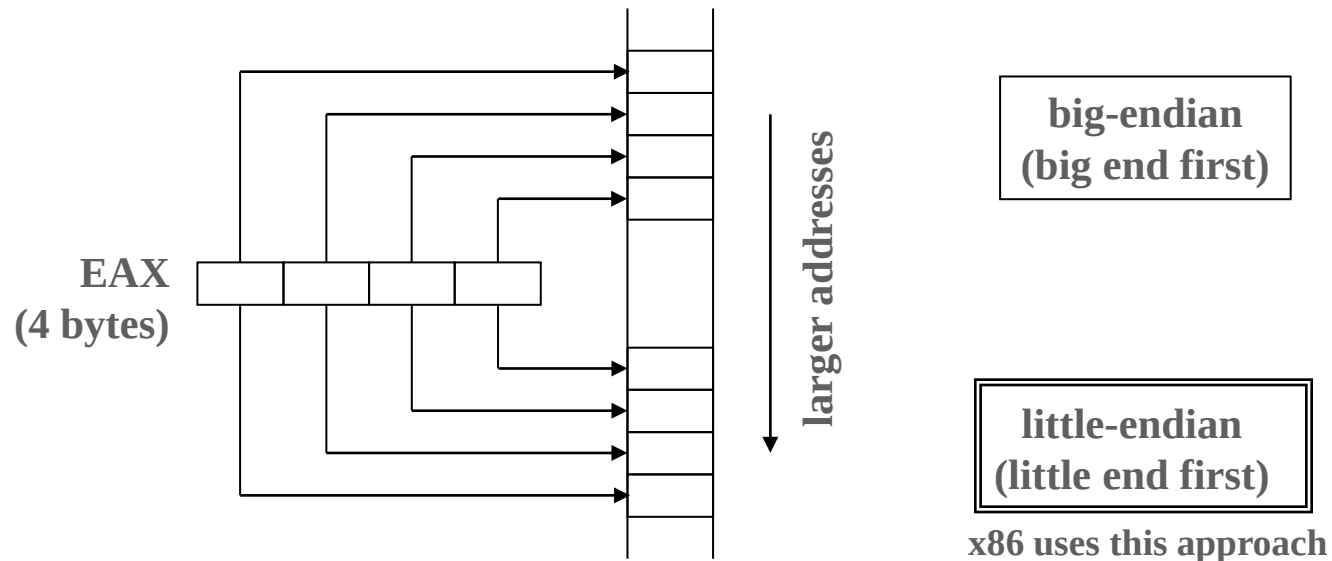
- Use % as a prefix for registers in assembly
- Question: can writing to AL, AH, or AX modify the whole EAX?
 - Hint: check Intel manual 3.4.1
- Other registers: FP, MMX, ... (not covered in this class)

Data Types

- 8-, 16-, 32-bit unsigned and 2's complement
- IEEE single- and double-precision floating point
- Intel “extended” f.p. (80-bit)
- ASCII strings
- Binary-coded decimal

- Refresher: how many memory locations can we address if memory address length is n bits.
 - Microprocessor addresses a maximum of 2^n different memory locations, where n is the number of bits on the address bus
- Memory
 - x86 supports byte addressable memory
 - byte (8 bits) is a basic memory unit
 - e.g., when you specify address 24 in memory, you get the entire eight bits
 - when the microprocessors address a 16-bit word of memory, two consecutive bytes are accessed

How are bytes stored to memory?



0x12345678 ➡ **0x78, 0x56, 0x34, 0x12**
in consecutive memory locations
(note: byte, not bit reversal)

x86 Instructions – Basics

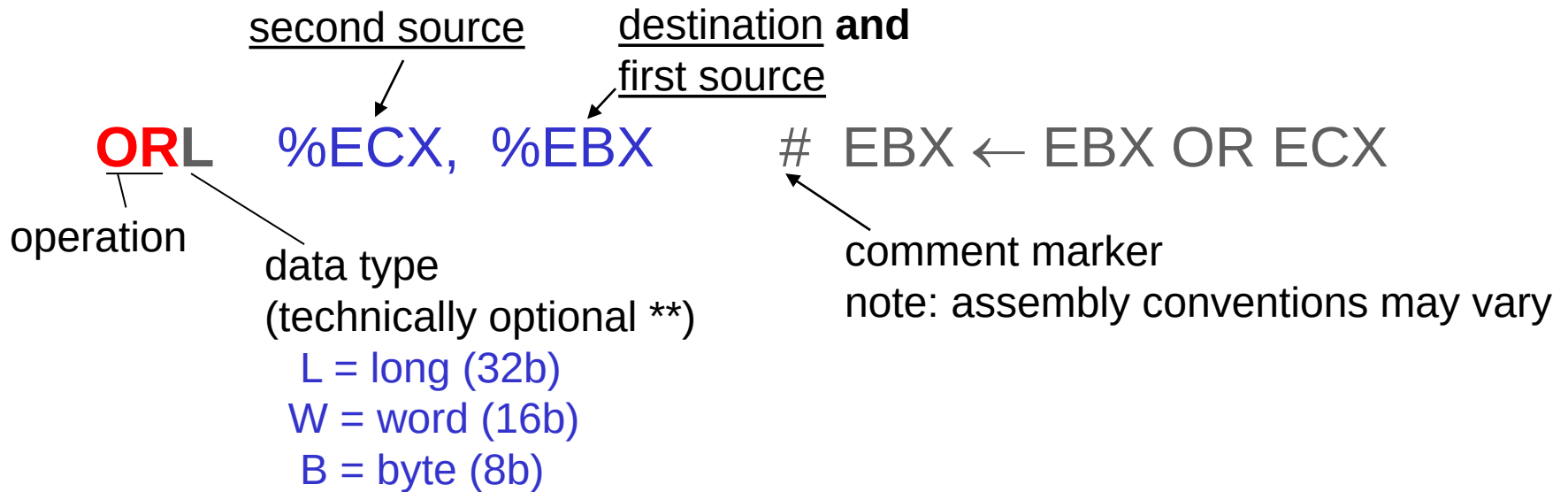
- Operations, data movement, condition codes, control flow, stack ops, data size conversion

Operations

<u>arithmetic</u>	<u>logical</u>	<u>shift</u>
ADD	AND	SHL
SUB	OR	SAR
NEG	NOT	SHR
INC	XOR	ROL
DEC		ROR

- Most instructions have two operands (destination and one source are the same)
 - Historical reasons, due to instruction encoding

Operations – Example



Immediate Values

immediate value marker

\$0x_____ hex

\$0_____ octal

\$53 decimal

1,2,9

what does the following instruction do?

ANDL 0, %EAX

answer is NOT
EAX ← 0

instead:
EAX ← EAX AND M[0]
(usually crashes)

- how big can they get?

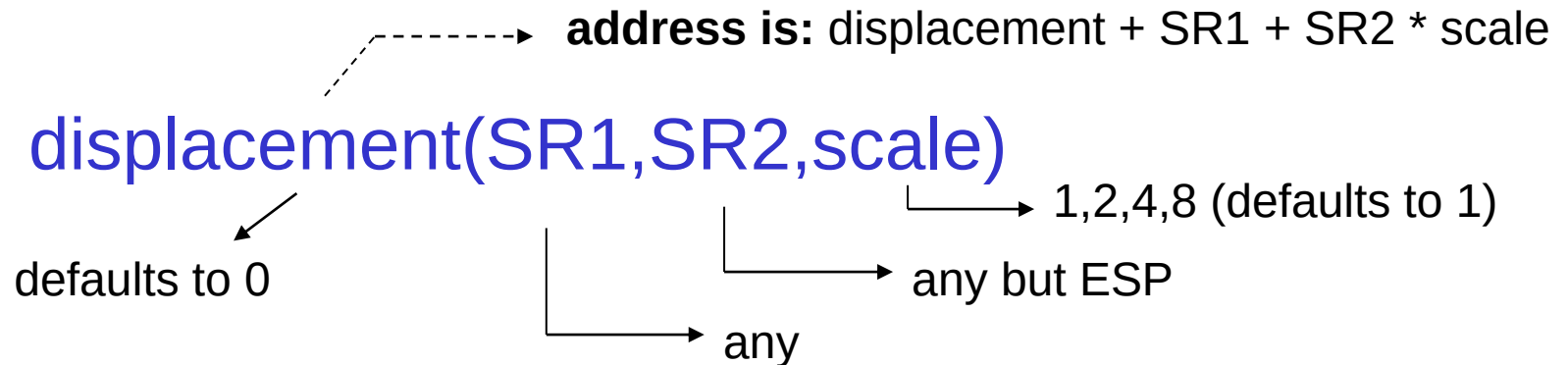
- usually up to 32 bits
- larger constants → longer instructions
- length of operand must be encoded, too

5-Minute Class Exercise

1. Write at least two ways of setting a register, let's say EAX, to zero. (use instructions learned so far, no MOV, LEA)
2. There is no register for the upper 16 bits of EAX, write instructions to store those bits in BX. EAX must remain unchanged after your code.
3. Write the body of a loop that computes Fibonacci numbers ($F_n = F_{n-1} + F_{n-2}$), assume EAX, EBX stores F_{n-1}, F_{n-2} initially, EDX should store current F_n , and ECX should be incremented each iteration. Don't worry about the loop's branch and termination.

Data Movement: Memory Addressing

Memory operand has this general form



Instructions

MOV $\swarrow$ immediate, register, or memory reference
 src, dst $\longrightarrow$ register, or memory reference

LEA src, dst $\longrightarrow$ register only
 $\swarrow$ memory reference only – address stored in dst

(can't both be memory references)

- Examples:

MOVW %DX, 0x10(%EBP) $\swarrow$ # M[EBP + 0x10] $\leftarrow$ DX

MOVB (%EBX,%ESI,4), %CL # CL $\leftarrow$ M[EBX + ESI * 4]

Instructions: Examples to Solve

$EAX \leftarrow M[0x10000 + ECX]$

[answer] **MOVL** 0x10000(%ECX) , %EAX

$M[LABEL] \leftarrow DI$

[answer] **MOVW** %DI , LABEL

$ESI \leftarrow LABEL + 4$ (two ways!)

[answer] **MOVL** \$LABEL + 4 , %ESI

LEAL LABEL + 4 , %ESI

$ESI \leftarrow LABEL + EAX + 4$

[answer] **LEAL** LABEL + 4(%EAX), %ESI

Assembler evaluated,
do not write 4(LABEL).
\$ sign treats address as
immediate.

Side Note

- When learning new assembly, try playing with a compiler.
- Or check out one online like: <https://godbolt.org/>
 - Set language to C and compiler to x86 gcc 1.27.

Condition Codes (in EFLAGS)

- Among others (not mentioned in this class)...
 - SF: sign flag: result is negative when viewed as 2's complement data type
 - ZF: zero flag: result is exactly zero
 - CF: carry flag: unsigned carry or borrow occurred
(or other, instruction-dependent meaning, e.g., on shifts)
 - OF: overflow flag: 2's complement overflow
(and other instruction-dependent meanings)
 - PF: parity flag: even parity in result (even # of 1 bits)

What Instructions Set Flags (condition codes)?

- Not all instructions set flags (e.g., MOV, LEA, NOT)
- Some instructions set some flags! (e.g., ROR, INC)

- Use **CMP** or **TEST** to set flags:

CMP**L** %EAX, %EBX # flags $\leftarrow$ (EBX – EAX)

TEST**L** %EAX, %EBX # flags $\leftarrow$ (EBX AND EAX)

- Note that EBX does not change in either case
- What combinations of flags are needed for unsigned/signed relationships comparator?

Control Flow Instructions (1)

- Consider two three-bit values A and B; How to decide if $A < B$?

A

B

C

CF

OF

SF

unsigned <

signed <

Control Flow Instructions (2)

- Note that CF suffices for unsigned $<$
- What about signed $<$?

- Answer: OF XOR SF

Branch Mnemonics

- Unsigned comparisons: “above” and “below”
- Signed comparisons: “less” and “greater”
- Both: equal/zero

unsigned	jne	jb	jbe	je	jae	ja
----------	-----	----	-----	----	-----	----

relationship	≠	<	≤	=	≥	>
--------------	---	---	---	---	---	---

signed	jne	j1	jle	je	jge	jg
--------	-----	----	-----	----	-----	----

- forms shown are those used when disassembling
 - do not expect binary to retain your version
 - e.g., “jne” becomes “jb”

Other Control Instructions

- Other branches
 - `jo` — jump on overflow (OF)
 - `jp` — jump on parity (PF)
 - `js` — jump on sign (SF)
 - `jmp` — unconditional jump
- **Control instructions:** subroutine call and return

`CALL printf` # (push EIP), $EIP \leftarrow \text{printf}$

`CALL *%EAX` # (push EIP), $EIP \leftarrow EAX$

`CALL *(%EAX)` # (push EIP), $EIP \leftarrow M[EAX]$

`RET` # $EIP \leftarrow M[ESP]$, $ESP \leftarrow ESP + 4$

Next Lecture: Stack Operations

- Next lecture: read ahead in course notes!
- Push and pop supported directly by x86 ISA

PUSHL %EAX # $M[ESP - 4] \leftarrow EAX, ESP \leftarrow ESP - 4$

POPL %EBP # $EBP \leftarrow M[ESP], ESP \leftarrow ESP + 4$

PUSHFL # $M[ESP - 4] \leftarrow EFLAGS, ESP \leftarrow ESP - 4$

POPFL # $EFLAGS \leftarrow M[ESP], ESP \leftarrow ESP + 4$